

Web Privacy Project

(ENGR-UH 4560) Spring 2025

Deadline for submission: 4 May 2025, 11:59 PM GST

Overview

Welcome to the Web Privacy project! The project consists of two parts. Part 1 involves analyzing a dataset to detect tracking behavior. Part 2 involves building your own privacy extension.

1. Analyzing an OpenWPM dataset

You read about OpenWPM in Week 5. In this project, you will be given access to a dataset collected using OpenWPM. The dataset consists of crawls to 177 sites. The crawler ran with two parallel Firefox browser instances, i.e., two sites crawled in parallel at a time. To keep the crawl data size small, we do not log source files of fetched resources or HTTP response bodies (so you might see NULL values in some of the columns).

1. Download the dataset [here](#). The dataset is 1.2 GB.
 - a. Note: If you have constraints downloading and using this dataset on your machine, please contact Sandra at sandra.siby@nyu.edu and she will make relevant arrangements.
2. The dataset is in the form of a SQLITE file. If you want a GUI, you can use [DB Browser](#) to view and analyze the tables in the dataset.
3. A description of the tables and columns is below. **Note that this only describes tables/columns that would be relevant to the project. You should be able to answer all the questions solely based on the provided information.** For more information on other tables/columns, please check the [OpenWPM repository](#). You can go through the OpenWPM repository if you require more information on the crawl and format (you can also try running a small crawl of your own if it helps you understand the dataset better).

- a. **site_visits:** shows the list of crawled sites
 - i. visit_id: Unique identifier used to identify a visit to a site. Use this to get all traffic associated with a particular site.
 - ii. browser_id: As the crawl ran with parallel browser instances, this ID indicates the particular instance of the browser for a site visit.
 - iii. site_url: URL of the visited site.
 - iv. site_rank: The crawl used a list of sites as input. The rank just shows the position of the site on that list.

- b. **crawl_history:** gives information about the crawl status
 - i. visit_id: Unique identifier used to identify a visit to a site.
 - ii. command: OpenWPM's commands to direct the crawl. The relevant command is GetCommand (which fetches the site).
 - iii. arguments: Arguments to the command.
 - iv. command_status: Status of running a particular command.
 - v. error: Error message in case of command failure.

- c. **http_requests:** Information on the HTTP requests sent during site visits
 - i. visit_id: Unique identifier used to identify a visit to a site.
 - ii. url: URL being fetched by the request.
 - iii. top_level_url: URL we see in the address bar.
 - iv. method: HTTP method.
 - v. headers: HTTP request headers.
 - vi. request_id: Numerical identifier of the request.
 - vii. resource_type: What kind of resource is fetched by the request (e.g., image, script, etc.).
 - viii. time_stamp: Time stamp of request.

- d. **http_responses:** Information on the HTTP responses.
 - i. visit_id: Unique identifier used to identify a visit to a site.
 - ii. url: URL being fetched by the request.
 - iii. top_level_url: URL we see in the address bar.
 - iv. method: HTTP method.
 - v. response_status: HTTP response status.
 - vi. headers: HTTP response headers.
 - vii. request_id: Numerical identifier of the request.
 - viii. time_stamp: Time stamp of response.

- e. **http_redirects:** Information on the redirects.
 - i. visit_id: Unique identifier used to identify a visit to a site.
 - ii. old_request_url: URL from which redirect occurred.
 - iii. new_request_url: New URL to which the redirect occurred.

- iv. `response_status`: Status of redirect.
- v. `headers`: Headers of redirect response.
- vi. `time_stamp`: Time stamp of redirect.

- f. **javascript**: JavaScript APIs that ran during a site visit.
 - i. `visit_id`: Unique identifier used to identify a visit to a site.
 - ii. `script_url`: URL of script that made an API call.
 - iii. `document_url`: Context within which the script ran.
 - iv. `top_level_url`: URL we see in the address bar.
 - v. `call_stack`: Stack trace of the API call.
 - vi. `symbol`: API call.
 - vii. `operation`: What kind of operation the call did (get/set, etc.).
 - viii. `value`: Value returned by the API call.
 - ix. `arguments`: Arguments passed by the call.
 - x. `time_stamp`: Time stamp of API call.

4. Please answer the following questions after analyzing the dataset.

- a. How many sites failed to be crawled? What was the most common reason for crawl failure?

For the remaining questions, use only sites that had a successful crawl.

- b. Plot a distribution of third parties per site. Which site had the highest number of third parties? Which third party was present on the largest number of sites?
- c. Which script set the largest number of cookies via JavaScript (across all sites)? How many of the cookies set by this script were in a first-party context?
- d. Which cookie (i.e., name of the cookie) was the most commonly set using HTTP across all sites?
- e. Plot a distribution of the number of cookie syncs observed per site, i.e., x-axis is the number of cookie syncs, y-axis is the number of sites of which a particular number of syncs was observed. (Hint: a cookie sync involves sharing the cookie value in an HTTP URL. For simplicity, you need not consider sharing in other parts such as headers or response bodies. If a particular instance of cookie syncing has multiple parties receiving the cookie, count each of these as a separate one)
- f. Pick one kind of JS fingerprinting we learned about in class/readings. On how many sites did this fingerprinting API appear? Analyze the scripts that set this API. Are there co-occurrences of fingerprinting APIs (i.e., multiple fingerprinting APIs being called by the same script)? If so, what was the most popular API co-occurrence – which other API appeared most frequently with the API that you selected?

2. Build your own privacy extension

In this part of the project, you will develop your own privacy browser extension for Chrome.

Read the tutorials [here](#) to understand how to structure an extension project and run it in the browser.

Required functionality:

Tracker blocking: Your extension should block trackers on the page, using a block list. You are free to use one (or multiple) block lists among the ones we have read about in class. Note that the popular block lists usually use regex-based rules: either modify the rules to a different format that your extension will use or modify your extension to handle regex.

Fingerprint protection mode: Provide a pop-up UI that will enable users to enable or disable fingerprinting protection. Pick one of the fingerprinting APIs (e.g., Canvas, WebGL, AudioContext). On enabling fingerprinting protection, your extension should (1) display to the user that the use of your chosen API has been detected on the site (2) override the values that this API returns with a randomized one.

One additional feature: Develop one feature of your choice for this extension. A few ideas (for inspiration, your idea need not belong to this list) – visualization of tracking, a real-time privacy score for your browsing session, a customizable block list that allows users to upload their own rules.

Testing and evaluation:

Tracker blocking: Test on at least five sites, especially ones that are likely to have tracking (news, shopping, etc.). Use Chrome DevTools to verify that requests are blocked. Provide a sample screenshot of one site to show this.

Fingerprint protection: Write a small script (either in console or on your webpage at webprivacylabs.com) that runs fingerprinting. Check if your extension behaves as expected here. Provide a sample screenshot to show this. You can test this on actual sites as well – note that we do not care if the page breaks for this project. You can mention that in your report if you observe this behavior.

Additional feature: Provide appropriate evidence (e.g., screenshots) to show the functionality of this feature.

Stretch goals:

Implementing extra features beyond the requirements or having a very well-done additional feature will get you bonus points (amount of points will be decided based on creativity, complexity, and correctness of the feature).

3. Project Submission

Submit your code and report on Brightspace, latest by 4 May 2025 11:59 PM GST. The report can be a single PDF file covering both parts. There is no specific format for the report or code – as long as it is precise and covers the requirements below, it is fine.

Part 1: You need to submit your code and a brief description of how you found the answers to the questions.

Part 2: You need to submit your code and report. The report should contain a brief description of how you implemented your extension. It should also provide the results of your testing and evaluation.

4. Grading Rubric

Category	Points	Description
Answers to questions in part 1	20 pts	Correct answers and explanations. Partial points will be given if the methodology is correct, but the final answer is not.
Tracker blocking	20 pts	Successfully blocks third-party tracking. Shows evidence with testing.
Fingerprinting protection	20 pts	Successfully modifies the fingerprinting API value. Shows evidence with testing. Toggling the protection is easy to use.

Additional feature	20 pts	Successfully implements the chosen feature. Shows evidence with testing.
Code Quality	5 pts	Code is well-structured and documented.
Report	5 pts	The report clearly describes the steps taken to complete the project.
Bonus	TBD	Does the extension go beyond the basic requirements?
